EE284A Lab 1:
Interfacing with Sensors and Actuators

1 Introduction

Sensors allow an IoT node to interface with the real world and collect data about its environment. There are many types of sensors, including cameras, strain gauges, humidity sensors, microphones, and even buttons. By converting physical phenomena such as motion, temperature, light, sound, or pressure into electrical signals, sensors make it possible for embedded systems to observe and respond to their surroundings. They are the eyes and ears of IoT nodes.

Actuators allow an IoT node to engage with the real world by producing an output or taking an action. While sensors bring information into the system, actuators send commands back out into the environment. Their interfaces to the MCU are often similar to those used by sensors, such as General Purpose Input/Output (GPIO), though some actuators also require additional power-handling circuitry to drive higher-current loads. Examples of actuators include LEDs, motors, buzzers, relays, and displays.

Sensors and actuators connect to the MCU through GPIO pins. Digital I/O pins can pass one bit between the internal circuits of the MCU and the external world by being set high (3.3V) or low (0V). Some GPIOs are internally connected to analog-to-digital converters (ADCs). In this case, the GPIO receives an analog voltage and provides the processor with a corresponding digital value. Other GPIOs are connected to circuits that implement data buses. A data bus typically uses more than one GPIO, along with specific protocols, voltages, sequences, and timing rules, to transfer and organize more data than a single bit. These protocols allow data to be streamed between devices.

In this lab, we will examine three forms of in-node interconnects: I2C, SPI, and analog. We will use several sensors and actuators to explore how an MCU interfaces with the real world. In the first half of the lab, you will build a system that takes action based on measurements from two temperature sensors while learning the basics of interfacing with sensors and actuators. In the second half of the lab, you will use an IR beam sensor and a motor. You will learn how sensor data can be used to monitor an actuator and how to design a feedback loop that achieves stable system behavior.

2 Learning Objectives

After this lab, you will be proficient in

- Reading the datasheets for sensors and the MCU
- Choosing a wired communication type appropriate for your sensor and actuator
- Processing raw sensor data to get realistic results
- Communicating sensor data to an MCU and using it to make decisions
- Connecting actuators appropriately given power demands

3 Materials

- ESP32 V2 Feather
- Breadboard and jumper wires
- Analog temperature sensor TMP36
- BME688 breakout board
- Resistors (330 ohms)
- Red and Green LEDs
- IR sensor
- Capacitors (470 μF)
- Servo motor

4 Analog Temperature Sensor

Many sensors output analog (continuous) signals rather than digital signals. This is because the real world is analog, and so the properties that we measure (resistance, capacitance, light intensity, etc.) are analog in nature as well. To interface with our digital circuits, we must convert analog signals to digital. This is typically done using an analog-to-digital converter (ADC) that maps voltages to discrete digital values. The feather has many analog input pins, all of which are labeled with a pin number starting with "A" (A0-A5). The digital values are discretized from 0 to $2^n - 1$ where n is the bit depth of the ADC. For the ESP32, the ADC is 12-bit, so the largest digital number that is output by the ADC is 4095, which corresponds to the maximum measurable input, V_{ref}. For the purposes of this lab $V_{ref} = 2.1$V. The conversion between the digital number (DN) and the real-world voltage is

$$V = \frac{DN}{2^{n-1}} V_{ref}$$

In this section, we are using the Analog Devices TMP36 analog temperature sensor in the T-3 (TO-92) packaging. This is a three-pin bandgap temperature sensor whose output voltage varies with temperature. The datasheet for the temperature sensor is found here: https://www.analog.com/media/en/technical-documentation/data-sheets/TMP35_36_37.pdf.
Review the datasheet and find the following and take screenshots of each:

- The pinout for the package we are using (which pin corresponds to V_{in}, GND, etc.)
- The device's range of operating voltages
- The graph which shows how our desired output changes with our desired input
- The table that shows the scaling between output and input

Note that the temperature sensor has a correct orientation as shown on its pinout diagram and is shown from a bottom up view. Be careful as mixing up the power and ground can fry the sensor. Connect the sensor to 3.3 V (the feather pin marked 3 V) and ground, with the V_{out} pin connected to A2. Take a photo of your wiring.

You will now need to figure out, based on this datasheet and what you know about the 12-bit ADCs on the feather, an equation to convert between ADC values and temperature. Write an Arduino program which reads the **A2 pin** and prints to serial "The raw ADC value is [value], which converts to [value]V. The temperature is [above][below] 23C, in fact it is [actual temperature]". In order to do this, you will need to put the following lines in the setup loop to make sure that your ADC is reading with all 12-bits:

```
analogReadResolution(12);
```

The attenuation of the input to the ADC is set in the code. For this part, we want everyone to use 6 dB of attenuation. Put these lines at the beginning of your loop:

```
analogSetPinAttenuation(ADC_PIN, ADC_6db); // lock attenuation
delay(50);
```

You must read the ADC values using

```
analogRead(ADC_PIN);
```

Take a screenshot of your output to submit. Try changing the temperature value by breathing on the sensor or wrapping the sensor on your palm to warm it. In many situations, we set a threshold on sensor values to determine what information we want to communicate or actuate based on this data. For example, perhaps if the temperature goes above 23C, we would like our IOT device to turn on a fan or send a hot temperature alert to a different device. We will be actuating based on the temperature data in later sections of this lab.

Deliverables

1. Screenshots from the TMP36 datasheet: package pinout, operating voltage range, the output-voltage-vs-temperature graph, and the scaling table.
2. A photo of your TMP36 wiring on the breadboard.
3. The equation you derived to convert raw 12-bit ADC values to degrees Celsius.
4. Screenshots of your serial output showing the temperature reported as above and below your chosen threshold.
5. Screenshot of your complete Arduino script for the TMP36 temperature reader.
6. Run for at least one minute and report the maximum and minimum temperature observed and the delta between them.
7. Research one other type of temperature sensor. What physical property changes with temperature and how is that property measured?

Note that while the raw ADC to voltage conversion is a useful exercise, in future labs you can use:

```
analogReadMilliVolts(TMP_PIN);
```

which uses the chip's built-in calibration.

5 I2C and SPI

The Bosch BME688 is gas, humidity, pressure and temperature sensor. However, unlike the analog sensor you used previously, this one has already converted the data to a digital format (it has a built in ADC) and can communicate this data via either I2C or SPI. In this section, you will wire this sensor both ways (note: that the board has internal pull-ups and thus requires nothing but jumper wires to be connected to the feather). The following link shows how to wire these and also how to access a quick test program that will read out the data from all sensors

https://learn.adafruit.com/adafruit-bme680-humidity-temperature-barometic-pressure-voc-gas/arduino-wiring-test

You will need to install the "Adafruit BME680 Library" and all dependencies. Run the test example code for each setup and test that you are able to read similar data from both setups (take screenshots).

Next modify the script to only output temperature information and modify the following lines in setup:

```
bme.setTemperatureOversampling(BME680_OS_1X);
bme.setIIRFilterSize(BME680_FILTER_SIZE_0);
```

This sets the BME board to not do any onboard filtering or averaging so that we can see the raw temperature data. Run for at least a minute and record the max and min temperature you see.

Deliverables

8. Photo of your I2C wiring and screenshot of the serial monitor showing the sensor data.
9. Photo of your SPI wiring and screenshot of the serial monitor showing the sensor data. Does the data look the same as over I2C?
10. Explain the order of events for transmitting data in I2C vs. SPI (high-level overview of what occurs per transaction).
11. What was the max and min temperature you saw over a minute with the BME688? How does this compare to the TMP36 sensor?

6 Signal Processing

You may have noticed that the temperature of both sensors changes and fluctuates over time, despite being in a constant temperature environment. This is because, as in many sensors, there is noise in the system. Some of this noise comes from the ADC, which needs to turn an analog value into discrete digital steps. Other factors may include the noise of the circuitry, power supply fluctuations that impact output, and noise from other digital signals nearby. Because of this, we typically need to average and filter the data.

Choose a temperature sensor and wire it up as done in previous sections. Now run experiments where you run for 5000 samples and track the max, min, mean, and standard deviation (std) of the samples for both temperature sensors. Implement and record the results when you apply the following signal processing:

- No filtering
- A moving average filter with N = 50 samples
- A moving average filter with N = 500 samples
- A moving median of 50 samples
- A moving median of 500 samples
- An exponential moving average with alpha = 2/(50+1)
- An exponential moving average of alpha = 2/(500+1)

If you are not familiar with moving averages, moving medians or exponential moving averages, please refer to the following links: moving average and exponentially weighted moving average and moving median. The first link also has several varieties of exponential moving averages that you are welcome to explore, though in this lab, we will only be implementing a simple exponentially weighted average.

Deliverables

12. Screenshot(s) of your implementation of a moving average, moving median, and exponential moving average. Specify which sensor you used.
13. A table comparing the max, min, standard deviation, and mean across all processing methods and both N values.
14. What did you notice about how each metric changed across methods? Did the results match your expectations?
15. What are the advantages and disadvantages of each method? How did the N value impact your results? Why might you choose a higher N, and what are the downsides?

7 Basic Actuators

We have been exploring how a node can sense the world around it, but sometimes we also want to act based on those sensor readings. Actuators are devices that, upon command from the MCU, execute a physical motion or action. One of the most common types are Light Emitting Diodes (LEDs). While many actuators are power hungry, LEDs typically draw low enough power that they can be driven directly from an MCU's GPIO pins. However, it is always good to check that you are within the current draw of a part before attaching peripherals. First, view the datasheet for the LED linked here. Find a graph showing the forward voltage of the LED (take a screenshot) and approximate the forward voltage of this diode when it is on. The forward voltage is the voltage drop we will expect to see across the LED. Note that, even at approximately the same voltage, we can observe a wide range of current through the LED. An LED is a diode that produces a constant voltage drop at sufficient current. Look at the datasheet for the Espressif chip that is on the feather linked here. Find the current rating for using the GPIO pins as a source (hint, the GPIO pins are in the VDD3P3 power domains). What about when used to sink current? Take a screenshot of these limits. In order to ensure that we do not go over this current limit (in fact we want to stay significantly under it) we will put a 330 ohm resistor in series with the LED. Pick three GPIO pins and wire each of them with a resistor and the LED in series from GPIO to ground. Write a program that prompts the user via serial to input which pins they would like to turn on. For example, the user might enter 011 to turn on LEDs 2 and 3, or 101 to turn on LEDs 1 and 3.

Deliverables

16. Screenshot of the LED datasheet showing the forward-voltage curve and your estimate of forward voltage. What do you notice about the forward current on this curve?

17. Screenshot of the ESP32 datasheet showing the GPIO source and sink current limits. What would happen if the LED were connected directly to the GPIO with no resistor?

18. Screenshot of the serial monitor with the user entering a code that turns on LEDs 1 and 3 but not LED 2. Include a photo of the LEDs in that state.

8 Buttons

The feather comes with two buttons on it, one is the reset button, which you may have already been using to restart your programs. The other is a free button that is attached to GPIO 0. Thus, we can program it for whatever we would like to use. Buttons are another type of sensor that measures if they have been pressed. Sometimes, when you press a button and write code to look for one state change, it looks like it was pressed multiple times, even if it was only pressed once by you. This is because there are oscillations after the button is pressed that make contact multiple times. Just like other sensors, buttons require some signal processing. However, instead of averaging, debouncing is done. There are many ways to implement debouncing, but the simplest is to wait after a button push for a set period of time before trying to read the button again. This allows the button to settle. There are no deliverables in this section, but you will be asked to use a button in the next section, so debouncing is something to keep in mind if you see irregular button behavior.

9 Making Decisions and Taking Action

In this section, we will combine the parts of the previous sections to create a full system. The end goal is to have a system in which both the digital temperature sensor and analog temperature sensor are being read, filtered, and compared. Then, if the difference between the two sensors is above a threshold (say if you are touching one sensor and warming it up), the LED will turn on. The system should not start reading the sensors, turning on the LED, or any other action until you have pushed a button. You may choose how you wire your system, which filtering method you choose, and what your threshold will be. Take photos of the system.

Deliverables

19. Screenshot(s) of your complete system code.

20. Photo of the system with the LED off.

21. Photo of the system with the LED on, and a description of what you did to trigger it.

22. Did you notice a temperature delta between the two sensors even as they were both in room temperature environments? If so, why might this be the case?

23. Explain which bus wiring you used and why. Explain which signal processing method you used and why. Did you sample each sensor in batches or did you flip back and forth between them? Why? Describe any other significant design decisions you made.

10 Sensor and Actuation in a Closed Loop

In this part of the lab, you will explore how sensors and actuators can work together to build an interactive embedded system. You will begin by using an infrared (IR) sensor to detect changes in the environment and observe how these changes are reflected in the sensor's output. You will then control a servo motor as an actuator and examine how it responds to commands from the microcontroller.

By combining sensing and actuation, you will gradually develop a simple closed-loop system, where the device can adjust its behavior based on real-time measurements. This type of feedback-driven operation is a fundamental concept in IoT systems, where devices continuously monitor their environment and respond accordingly.

10.1 Setting up the IR sensor

In this subsection, you will assemble an infrared (IR) break-beam link and verify its operation using the provided demo code and the Serial Plotter. This link is composed of an IR emitter and an IR receiver. If an object or person passes through the beam, the link will be interrupted. This can be used for counting objects/people, detecting motion, checking if a door is open or closed, and many other things. Refer to this link for more details Link.

To setup the sensor, first, use tape to mount the IR transmitter and IR receiver so that they face each other directly. The two components should be aligned along the same axis, with a separation distance of approximately 10–30 cm. Proper alignment is important for reliable detection. An example mounting is shown in Figure 1.

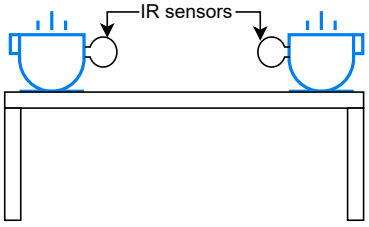

Figure 1 :
Example mounting. You are encouraged to develop your own mounting strategies.

Next, connect the IR receiver output (yellow wire) to `GPIO 27`. Power the sensor by connecting VCC to 3.3 V and GND to ground, then run the following demo code.

```
const int beamPin = 27;

void setup() {
  Serial.begin(115200);
  pinMode(beamPin, INPUT_PULLUP);
}

void loop() {
  int state = digitalRead(beamPin);
  if (state == HIGH) {
    Serial.println(1);
  } else {
    Serial.println(0);
  }
  delay(10);
}
```

Open the Serial Monitor or Serial Plotter and set the baud rate to `115200`. Observe the output values while the beam is unobstructed, and then again while the beam is blocked with your hand.

Deliverables

24. Submit a photo showing the IR sensor setup and the Serial Plotter output while your hand briefly interrupts the IR beam.

10.2 Controlling the Motor

In this subsection, you will drive a servo motor using a PWM signal and observe its rotational behavior. The servo will act as the actuator in the system, responding to control commands generated by the microcontroller.

Refer to the datasheet when doing the wiring. First, connect the servo motor control signal wire (yellow wire) to the designated PWM-capable GPIO pin, e.g., `GPIO 13`. Connect the servo ground (black wire) to GND and the power line (red wire) to a suitable 5 V supply. If the microcontroller is powered from a laptop via USB-C, the USB pin on the board provides 5 V. Add a polarized 470 μF capacitor between 5 V and ground, ensuring the correct polarity: the longer lead (positive) should be connected to 5 V, and the shorter lead (negative), should be connected to ground. Reversing the polarity may damage the capacitor and could lead to overheating or failure.

The following code configures the ESP32 to generate a 50 Hz PWM signal using a 16-bit resolution timer. The helper function `dutyFromUs()` converts a desired pulse width (in microseconds) into the corresponding duty cycle value required by the hardware PWM peripheral.

```
const int servoPin = 13;
const int pwmResolution = 16; // 16-bit resolution
const int pwmFreq = 50;
uint32_t minPulse = 1550;
uint32_t midPulse = 1650;
uint32_t maxPulse = 1750;

// Convert pulse width in microseconds to duty cycle value
uint32_t dutyFromUs(uint32_t pulseUs) {
  const uint32_t maxDuty = (1UL << pwmResolution) - 1;
  return (pulseUs * maxDuty) / 20000UL;
```

```
}

void setup() {
  ledcAttach(servoPin, pwmFreq, pwmResolution);
  delay(100);
  ledcWrite(servoPin, dutyFromUs(maxPulse));
}

void loop() {
}
```

In this example, the PWM signal is generated on `GPIO 13`. The line

```
ledcWrite(servoPin, dutyFromUs(midPulse));
```

sets the pulse width sent to the servo motor. Try out three different pulse widths. This step establishes the actuator side of the system. In the following section, the motor motion will be regulated using feedback from the IR sensor to achieve a stable target frequency.

Deliverables

25. Explain the role of the capacitor for 5V supply. You may experiment by removing it.

10.3 Putting the Sensor and Actuator Together

In this subsection, you will integrate the actuator (servo motor) and the sensor (IR break-beam sensor) to build a closed-loop speed control system.
This subsection has two steps: (1) measure motor speed (RPM) using the IR sensor, and (2) use the measured RPM to form a feedback loop that regulates the motor to a target speed of 60 RPM.

Step 1: Measure the Motor Speed (Open Loop). Set up the IR sensor so that the rotating part interrupts the IR beam periodically. A typical approach is to attach a small piece of tape (or a printed flag) to the rotating shaft/horn so that it passes through the beam once per revolution. Be creative!
Your task is to estimate RPM from the IR sensor signal. Detect beam events (edges) in software and measure the time between events. You can use function `micros()` to obtain the current time in microseconds.

Deliverables

26. Convert the interval to RPM. Measure and report the estimated RPM for the three pulse widths you used in the previous subsection.

Step 2: Closed-Loop Speed Control (Target = 60 RPM). Next, use the RPM measurement from Step 1 to implement a feedback loop that regulates the motor speed to a target of 60 RPM.

Defining a target speed `rpmTarget` = 60, your controller should work in this loop:

- Wait for a new speed estimation `rpmMeasured` from the IR sensor.
- Compute an error signal $e =$ `rpmTarget` $-$ `rpmMeasured`.
- Update the PWM pulse width command based on the e.

You may implement any reasonable feedback strategy. For example, incremental control, proportional control or even PID control. No matter what strategy you choose, always remember to clamp the pulse width to a safe range. In our case, it should not be outside the interval [`minPulse`, `maxPulse`]. Tune your controller until the measured speed settles near 60 RPM with minimal oscillation.

Deliverables

27. Briefly describe the final controller design you implemented.

28. Record a short video that shows the motor, the IR sensor setup, and the Serial Monitor/Plotter RPM output, beginning from when the program starts running. Attach the link of the video in your report.

Lastly, don't forget to include a link to your code!